

DOCUMENTAÇÃO PLANO DE TESTES: PROJETO FINAL - MODERNIZAÇÃO DE UM SISTEMA LEGADO

Autor: Matheus Martins da Cruz

1. CASOS DE TESTE

Os casos de teste é referente a validar a segurança do código, permitindo a entrada de dados apenas se forem válidas.

- Primeiro caso: Consulta de cliente (Válido)
 - Critério: consulta um cliente válido no Swagger, recebe o id repassa para o COBOL, acha o cliente e ele lê os dados no arquivo CLIENTES.
- Segundo caso: Consulta de cliente (Inválida)
 - Critério: consulta um cliente inválido no Swagger, recebe o id repassa para o COBOL, ele procura o id no arquivo CLIENTES.
- Terceiro caso: Cadastrar cliente (Válido)
 - Critério: cadastra um cliente válido no Swagger, recebe os dados pelo Request e repassa para o COBOL, ele escreve os dados no arquivo CLIENTES.
 -
- Quarto caso: Cadastrar cliente (Inválido)
 - Critério: cadastra um cliente inválido no Swagger, o usuário tenta cadastrar sem o nome que é obrigatório.
- Quinto caso: Atualizar cliente (Válido)
 - Critério: atualiza um cliente válido no Swagger, recebe o id e os dados pelo Request e repassa para o COBOL, ele lê e encontra o cliente, depois reescreve os dados no arquivo CLIENTES.
- Sexto caso: Atualizar cliente (Inválido)
 - Critério: atualiza um cliente inválido no Swagger, o usuário tenta atualizar um cliente que não existe no arquivo, o COBOL recebe o id e os dados, ele lê o id e procura no arquivo CLIENTES e não encontra.

- Sétimo caso: Deletar cliente (Válido)
 - Critério: deleta um cliente válido no Swagger, recebe o id e repassa para o COBOL, procura o cliente e deleta no arquivo CLIENTES.
- Oitavo caso: Deletar cliente (Inválido)
 - Critério: deleta um cliente inválido no Swagger, recebe o id repassa para o COBOL, ele procura o id no arquivo CLIENTES.

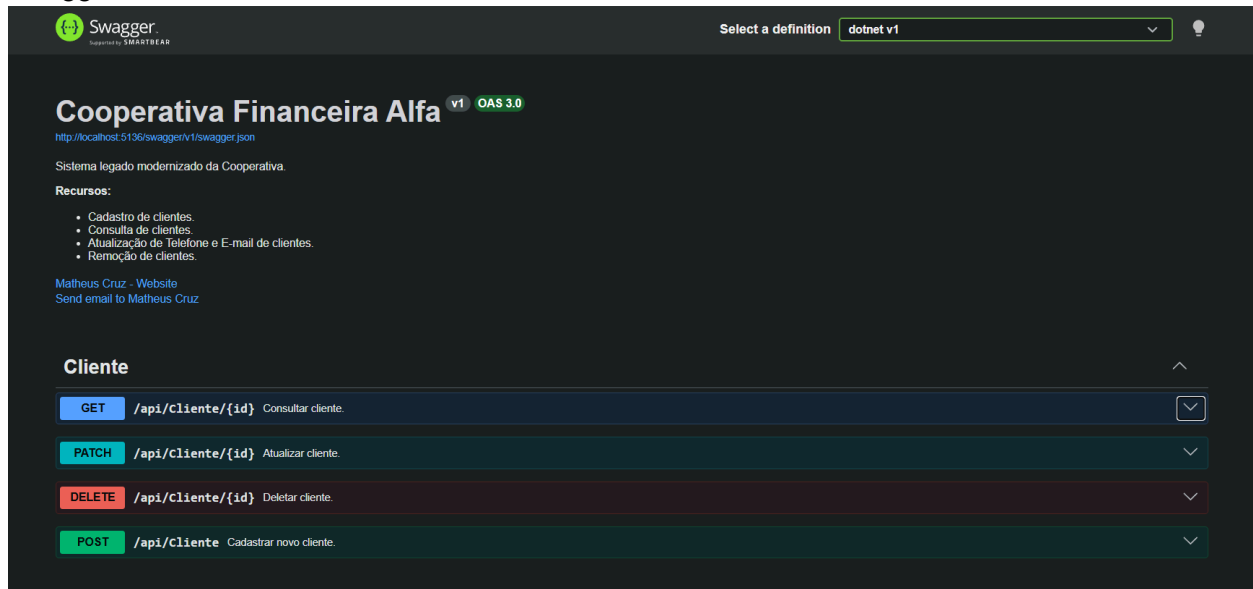
2. CRITÉRIOS DE ACEITAÇÃO

- Primeiro caso: Consulta de cliente (Válido)
 - Critério de sucesso: O sistema retorna status 200 OK com os dados do cliente carregados do VSAM.
- Segundo caso: Consulta de cliente (Inválido)
 - Critério de sucesso: O sistema retorna status 404 Not Found informando que o cliente não foi encontrado.
- Terceiro caso: Cadastrar cliente (Válido)
 - Critério de sucesso: O sistema grava os dados no VSAM e retorna status 200 OK.
- Quarto caso: Cadastrar cliente (Inválido)
 - Critério de sucesso: O sistema valida a ausência do dado e retorna status 400 Bad Request.
- Quinto caso: Atualizar cliente (Válido)
 - Critério de sucesso: O sistema localiza o registro, atualiza as informações no VSAM e retorna status 200 OK.
- Sexto caso: Atualizar cliente (Inválido)
 - Critério de sucesso: O sistema retorna status 404 Not Found.

- Sétimo caso: Deletar cliente (Válido)
 - Critério de sucesso: O sistema remove o registro do VSAM e retorna status 204 No Content.
- Oitavo caso: Deletar cliente (Inválido)
 - Critério de sucesso: O sistema retorna status 404 Not Found.

3. EVIDÊNCIAS DA EXECUÇÃO

Swagger documentado com XML



- Primeiro caso
antes da execução:

Cliente

GET

/api/Cliente/{id}

Consultar cliente.

Observação:

- ID é obrigatório, tamanho máximo de 5 dígitos

Parameters

Name

Description

id * required

ID do cliente a ser consultado.

Integer(\$int32)

(path)

4

Cancel

Execute

Responses

Code	Description	Links
200	Cliente encontrado com sucesso!	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

depois da execução:

ID * required

ID do cliente a ser consultado.

integer(int32)

(path)

4

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5136/api/Cliente/4' \
-H 'accept: text/plain'
```

Request URL

```
http://localhost:5136/api/Cliente/4
```

Server response

Code

Details

200

Response body

```
{
  "cli Id": 4,
  "cli Nome": "MATEUS",
  "telefone": "(34)97777-9999",
  "email": "matheuzin@gmail.com"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed, 08 Jul 2026 16:22:47 GMT
server: Kestrel
transfer-encoding: chunked
```

- Segundo caso:

antes da execução:

Cliente

GET

/api/Cliente/{id}

Consultar cliente.

Observação:

- ID é obrigatório, tamanho máximo de 5 dígitos

Parameters

Cancel

Name	Description
id * required integer(int32) (path)	ID do cliente a ser consultado.

77

Execute

depois da execução:

id * required
integer(int32)
(path)

ID do cliente a ser consultado.

77

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:5136/api/Cliente/77' \
-H 'accept: text/plain'
```

Request URL

http://localhost:5136/api/Cliente/77

Server response

Code	Details
404	Error: Not Found Response body { "erro": "Cliente não encontrado" } Response headers content-type: application/json; charset=utf-8 date: Wed, 08 Jul 2026 16:24:40 GMT server: Kestrel transfer-encoding: chunked

Download

- Terceiro caso:

antes da execução:

POST /api/Cliente Cadastrar novo cliente.

Observações:

- Nome é obrigatório, tamanho máximo de 30 letras
- Telefone tem tamanho máximo de 15 caracteres
- E-mail tem tamanho máximo de 30 caracteres

Exemplo de requisição:

```
POST /api/clientes
{
  "cli_Nome": "Aline Dutra",
  "telefone": "(34) 97777-2222",
  "email": "aline@aceleramaker.com.br",
}
```

Parameters Cancel Reset

No parameters

Request body application/json

Dados do cadastro a ser criado.

[Edit Value](#) | [Schema](#)

```
{
  "cli_Nome": "Luis Guerreiro",
  "telefone": "(11) 91111-4444",
  "email": "luis@aceleramaker.com.br"
}
```

CONTROLE.dat ✕

COBOL > DATA > **CONTROLE.dat**

You, yesterday | 1 author (You)

1	00012
2	

depois da execução:

Responses

Curl

```
curl -X 'POST' \
  http://localhost:5136/api/Cliente' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "cli_Nome": "Luis Guerreiro",
    "telefone": "(11) 91111-4444",
    "email": "luis@aceleramaker.com.br"
  }'
```

Request URL

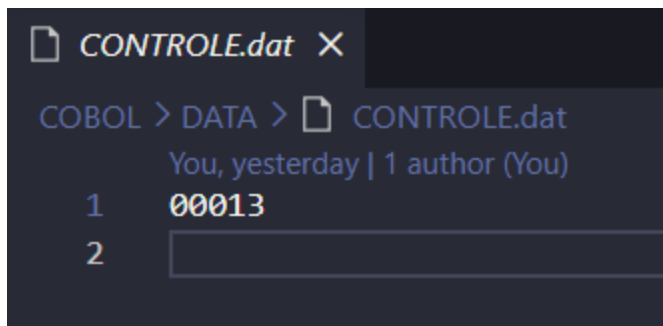
http://localhost:5136/api/Cliente

Server response

Code	Details
201	Response body <pre>{ "cli_id": 13, "cli_Nome": "Luis Guerreiro", "telefone": "(11) 91111-4444", "email": "luis@aceleramaker.com.br" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 08 Jul 2026 16:32:15 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
201	Cadastro criado com sucesso!	No links



- Quarto caso:

antes da execução:

POST

/api/Cliente

Cadastrar novo cliente.

Observações:

- Nome é obrigatório, tamanho máximo de 30 letras
- Telefone tem tamanho máximo de 18 caracteres
- E-mail tem tamanho máximo de 30 caracteres

Exemplo de requisição:

```
POST /api/clientes
{
  "cli_Nome": "Aline Dutra",
  "telefone": "(21) 97777-2222",
  "email": "aline@aceleramaker.com.br",
}
```

Parameters

No parameters

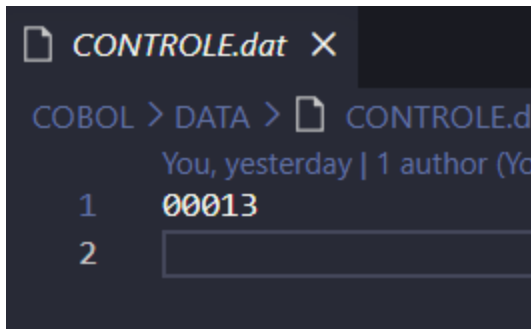
Request body

application/json

Dados do cadastro a ser criado.

Edit Value | Schema

```
{
  "cli_Nome": "",
  "telefone": "(21) 98888-9999",
  "email": "semmnome@email.com"
}
```



depois da execução:

```
curl -X 'POST' \
  http://localhost:5136/api/Cliente \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "cli_Nome": "",
    "telefone": "(21) 98888-9999",
    "email": "semmnome@email.com"
  }'
```

Request URL

http://localhost:5136/api/Cliente

Server response

Code	Details
400	Error: Bad Request

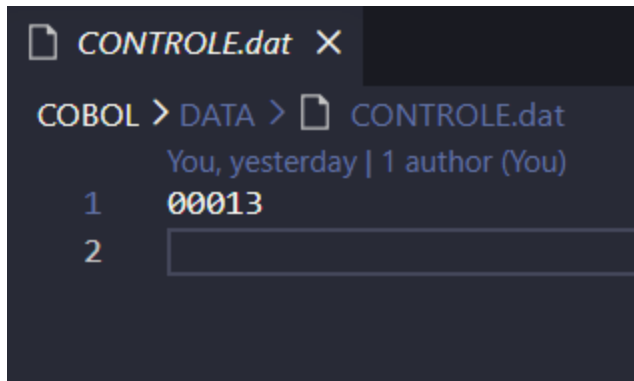
Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "cli_Nome": [
      "Nome é obrigatório.",
      "O nome deve ter entre 1 e 30 caracteres."
    ]
  },
  "traceId": "00-9b81db75935c9440bd78b372224ac7cf-07efb09bc36cab2c-00"
}
```

Response headers

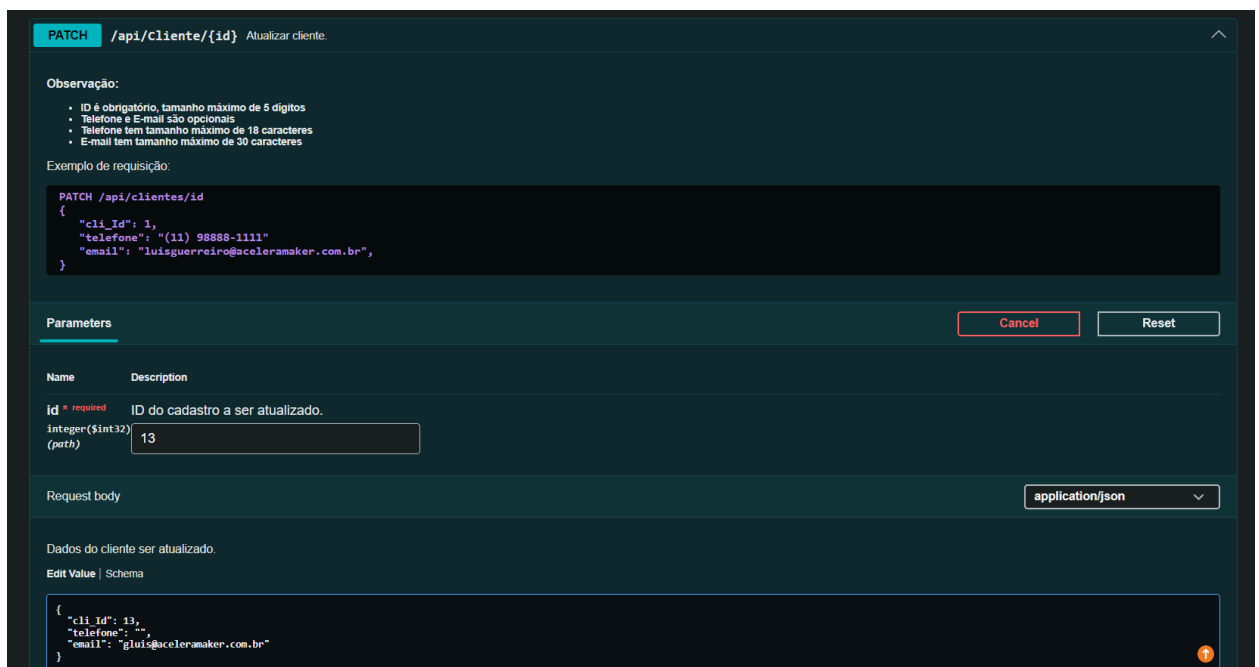
```
content-type: application/problem+json; charset=utf-8
date: Wed, 08 Jul 2020 16:35:27 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses



- Quinto caso:

antes da execução:



depois da execução:

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:5136/api/Cliente/13' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "cli_id": 13,
    "telefone": "",
    "email": "gluis@aceleramaker.com.br"
  }'
```

Request URL

http://localhost:5136/api/Cliente/13

Server response

Code	Details
200	Response body <pre>{ "cli_id": 13, "cli_nome": "Luis Guerreiro", "telefone": "(11) 91111-4444", "email": "gluis@aceleramaker.com.br" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 08 Jul 2026 16:39:21 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	Cliente atualizado com sucesso!	No links

• Sexto caso:

antes da execução:

PATCH /api/Cliente/{id} Atualizar cliente.

Observação:

- ID é obrigatório, tamanho máximo de 5 dígitos
- Telefone e E-mail são opcionais
- Telefone tem tamanho máximo de 18 caracteres
- E-mail tem tamanho máximo de 30 caracteres

Exemplo de requisição:

```
PATCH /api/clientes/id
{
  "cli_id": 1,
  "telefone": "(11) 98888-1111",
  "email": "luisguerreiro@aceleramaker.com.br",
}
```

Parameters

Name	Description
id * required integer(\$int32) (path)	ID do cadastro a ser atualizado.

Request body

application/json

Dados do cliente ser atualizado.

Edit Value | Schema

```
{
  "cli_id": 13,
  "telefone": "45345435345435345534435543345435345",
  "email": "guerreiroluis@aceleramaker.com.br"
}
```

depois da execução:

```
Curl
curl -X 'PATCH' \
  'http://localhost:5136/api/Cliente/13' \
  -H 'accept: text/plain' \
  -H 'content-type: application/json' \
  -d '{
    "cli_id": 13,
    "telefone": "45345435345435345534435543345435345",
    "email": "guerreiroluis@aceleramaker.com.br"
  }'
```

Request URL

http://localhost:5136/api/Cliente/13

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "email": [
      "O e-mail deve ter entre 1 e 30 caracteres."
    ],
    "telefone": [
      "O telefone deve ter entre 1 e 18 caracteres."
    ]
  },
  "traceId": "00-b777f413084a2788056c7fe58d86085b-9d4997a6d82ec37f-00"
}
```

Response headers

```
content-type: application/problem+json; charset=utf-8
date: Wed, 06 Jul 2026 16:41:06 GMT
server: Kestrel
transfer-encoding: chunked
```

- Sétimo caso:

antes da execução:

DELETE /api/cliente/{id} Deletar cliente.

Observação:

- ID é obrigatório, tamanho máximo de 5 dígitos

Parameters

Cancel

Name	Description
id * required integer(\$int32) (path)	ID do cadastro a ser deletado.

13

Execute

Responses

Code	Description	Links
204	Ciente deletado com sucesso!	No links

Media type
text/plain
Controla Accept header

depois da execução:

Name	Description
id * required integer(\$int32) (path)	ID do cadastro a ser deletado.

Execute
Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5136/api/Cliente/13' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5136/api/Cliente/13

Server response

Code	Details
204	Response headers <pre>date: Wed, 08 Jul 2026 16:41:58 GMT server: Kestrel</pre>

Responses

Code	Description	Links
204	Cliente deletado com sucesso!	No links

- Oitavo caso:

antes da execução(tentar excluir o mesmo):

DELETE
/api/cliente/{id}
Deletar cliente.

Observação:

- ID é obrigatório, tamanho máximo de 5 dígitos

Parameters

Cancel

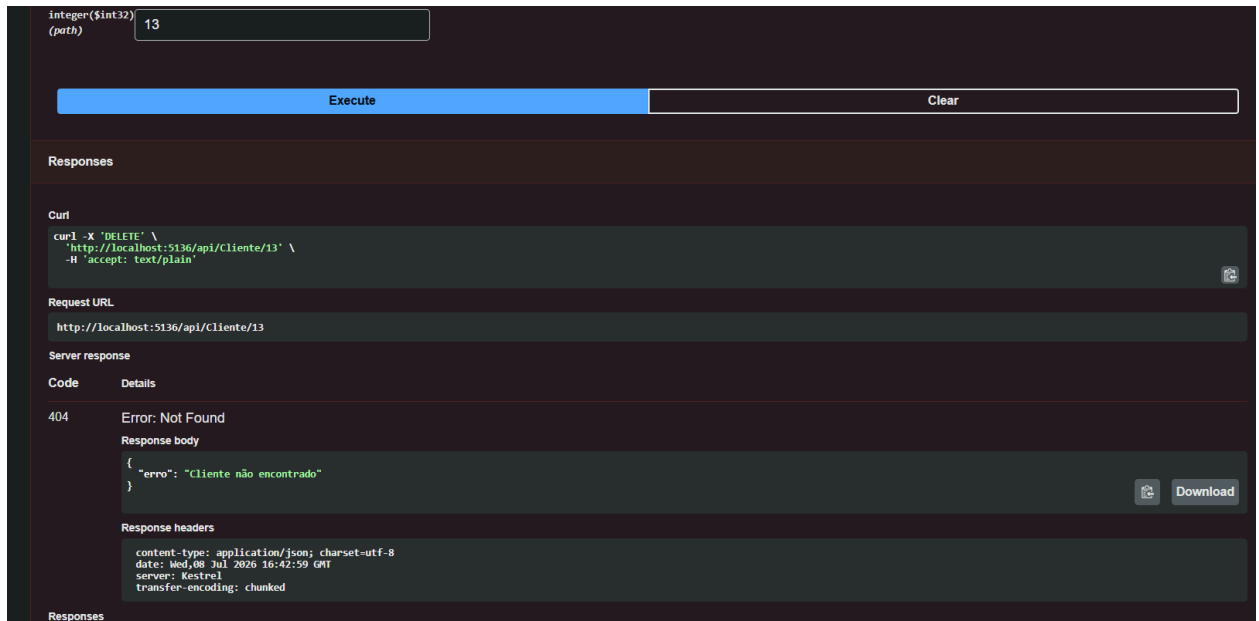
Name	Description
id * required integer(\$int32) (path)	ID do cadastro a ser deletado.

Execute

Responses

Code	Description	Links
204	Cliente deletado com sucesso!	No links

depois da execução:



4. TESTES AUTOMATIZADOS

Foram implementados 12 testes, 8 parecidos com os testes de caso acima, porém implementados pelo XUnit e 4 testes referentes ao CopyBook.

Todos estão na pasta “tests” dentro da pasta “CodigoFonte”, vou colocar print de dois, porém o resto segue a mesma lógica, e também a confirmação que foram validados.

O primeiro é um teste para verificar se o wrapper preenche com espaços corretamente

```

1  [Fact]
2  public void PayloadCobol_PreencheEspacos()
3  {
4      //ARRANGE
5      var parser = new CopybookParser(_caminhoCopybook);
6      var wrapper = new CopybookWrapper(parser);
7      //simulando a requisição enviada pelo usuario
8      string nome = "MATHEUS";
9      string telefone = "11999999999";
10     string email = "teste@teste.com";
11
12     //ACT
13     wrapper.PayloadCobol("0", nome, telefone, email, "");
14
15     //ASSERT
16     string nomeLimpo = wrapper.ExtrairCampo(1);
17     //nome sem espaços no final
18     Assert.Equal("MATHEUS", nomeLimpo);
19
20     string memoriaNome = System.Text.Encoding.ASCII.GetString(wrapper.BufferMemoria, 5, 30);
21     //nome com espaços no final e de tamanho 30
22     Assert.Equal(30, memoriaNome.Length);
23     Assert.EndsWith(" ", memoriaNome);
24 }

```

E o segundo é para testar se é possível cadastrar cliente sem nome

```

1  [Fact]
2  public void Cadastar_Cliente_Invalido()
3  {
4      //ARRANGE
5      var parser = new CopybookParser(_caminhoCopybook);
6      var wrapper = new CopybookWrapper(parser);
7      wrapper.PayloadCobol("0", " ", "(11) 98888-7777", "luis@email.com", "TS");
8      //enviado sem o nome
9
10     //ACT
11     DllConfigTests.cob_init(0, IntPtr.Zero);
12     DllConfigTests.CADASCLI(wrapper.BufferMemoria);
13
14     //ASSERT
15     //99 - erro nome é obrigatório
16     string statusRetorno = wrapper.ExtrairCampo(4).Trim();
17     Assert.Equal("99", statusRetorno);
18 }
19

```

Obsevarção: para que deixar organizado o arquivo CLIENTES e o arquivo de testes CLITESTS, é passado como parâmetro no StatusRetorno como "TS" (de Testes), nessa linha `wrapper.PayloadCobol("0", " ", "(11) 98888-7777", "luis@email.com", "TS");` assim ele acessa os arquivos de testes e não altera o arquivo de CLIENTES.

- Confirmação de testes aprovados:
 - antes de rodar o comando

```
D:\AceleraMaker\projetosAceleraMaker\ProjetoFinal\CodigoFonte\dotnet>cd ../tests
```

```
D:\AceleraMaker\projetosAceleraMaker\ProjetoFinal\CodigoFonte\tests>dotnet test
```

- depois de rodar o comando

```
D:\AceleraMaker\projetosAceleraMaker\ProjetoFinal\CodigoFonte\dotnet>cd ../tests

D:\AceleraMaker\projetosAceleraMaker\ProjetoFinal\CodigoFonte\tests>dotnet test
Restauração concluída (0,9s)
dotnet net10.0 êxito (0,7s) → D:\AceleraMaker\projetosAceleraMaker\ProjetoFinal\CodigoFonte\dotnet\bin\Debug\net10.0\dotnet.dll
tests net10.0 êxito (5,2s) → bin\Debug\net10.0\tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.12] Discovering: tests
[xUnit.net 00:00:00.18] Discovered: tests
[xUnit.net 00:00:00.21] Starting: tests
[xUnit.net 00:00:01.07] Finished: tests
tests teste net10.0 êxito (2,3s)

Resumo do teste: total: 12; falhou: 0; bem-sucedido: 12; ignorado: 0; duração: 2,3s
Construir êxito em 10,1s

D:\AceleraMaker\projetosAceleraMaker\ProjetoFinal\CodigoFonte\tests>
```